



Graf Algoritmaları ve Trace

Problem - Graf Eşleştirmesi

1. Ulaşılabilirlik / Bağlantı Kontrolü (Connectivity / Reachability)

- **Soru şu:** "Ben A şehrinden kalkıp B şehrine raylar üzerinden gidebilir miyim, arada yol var mı?"
- **Çalışma Süresi:** $O(V + E)$. Yani şehir sayısı + yol sayısı kadar çok kısa bir sürede biter.
- **Uygun Algoritmalar:** **BFS (Breadth-First Search)** veya **DFS (Depth-First Search)**.
 - **BFS (Genişlik Öncelikli):** Bulunduğun istasyondan başlayarak dalga dalga, önce en yakınındaki tüm komşu istasyonlara uğrar, sonra onların komşularına geçer (tıpkı suya atılan taşın oluşturduğu halkalar gibi).
 - **DFS (Derinlik Öncelikli):** Bir tünele girip yol çatallansa bile inatla o yolun en sonuna kadar gider, çıkmaza girdiğinde geri dönüp (backtrack) diğer çatalı dener.

2. Ağırlıksız Graflarda En Kısa Yol

- **Soru şu:** "Yolların uzunlukları (maliyetleri) eşit veya önemsiz. Ben A istasyonundan B istasyonuna **en az sayıda istasyondan geçerek** nasıl giderim?"
- **Çalışma Süresi:** $O(V + E)$
- **Uygun Algoritmalar:** **BFS (Breadth-First Search=**

- BFS dalga dalga ve adım adım yayıldığı için, hedef istasyonu gördüğü ilk an, oraya giden en az adımlı (en kısa) yolu bulduğu andır. Sınavda ağırlıksız derse direkt BFS'yi yapıştır.

3. Ağırlıklı Graflarda En Kısa Yol (No Negative Edges)

- **Soru şu:** "Şehirler arası yolların kilometreleri (ağırlıkları) var ve hepsi pozitif (+5 km, +10 km gibi). A'dan B'ye giden **en kısa kilometreli** yol hangisi?"
- **Çalışma Süresi:** $O((V + E) \log V)$ (Binary heap veri yapısı kullanıldığında)
- **Uygun Algoritmalar:** Dijkstra
 - Tıpkı telefonundaki Google Haritalar / Navigasyon uygulaması gibidir.
 - Bulunduğun düğümden dışarı doğru en küçük maliyetli yolları seçerek ilerler ve ulaştığı düğümlerin mesafesini günceller.
 - **Çok önemli kural:** Grafin içinde negatif ağırlıklı (eksi değerli) bir yol varsa Dijkstra kör olur, yanlış çalışır.

4. Negatif Kenarlı Graflarda En Kısa Yol

- **Soru şu:** "Yolların bazılarında geçerken benzin harcamıyorsun, aksine sponsor sana para ödüyor (Yani yolun maliyeti -5 TL gibi negatif bir değer). En ucuz yolu bul."
- **Çalışma Süresi:** $O(V \cdot E)$. (Dijkstra'ya göre daha yavaştır çünkü her ihtimali döne döne kontrol eder)
- **Uygun Algoritma:** Bellman-Ford
 - Grafin içinde tüm kenarları $V - 1$ defa tek tek kontrol ederek mesafeleri günceller. Ayrıca grafin içinde sonsuza kadar dönüp para basabileceğin "Negatif Çevrim (Negative Cycle)" olup olmadığını da bir ekstra tur atarak tespit eder.

5. Minimum Yayılan Ağaç (MST)

- **Soru şu:** "Elimde 10 tane köy var. Bu köylerin hepsine elektrik şebekesi veya internet kablosu döşemek istiyorum. Köylerin hepsi bir şekilde birbirine

bağlansın (ulaşılabilir olsun) ama **kullanacağım toplam kablo uzunluğu minimum olsun** ve hatlar arasında elektrik kısır döngüsü (cycle/çevrim) oluşmasın".

- **Çalışma Süresi:**

- Graf çok yoğunsa (yol çoksa) Prim $O(V^2)$,
- graf seyrekse (yol azsa) Kruskal veya heap tabanlı Prim $O(E \log E)$ veya $O(E \log V)$ çalışır.

- **Uygun Algoritmalar:**

- **Prim:** Rastgele bir köyden başlar, o köye bağlanan en ucuz kabloyu seçerek ağacı adım adım büyütür.
- **Kruskal:** Grafın içindeki tüm yolları en ucuzdan en pahalıya doğru sıraya dizer. En ucuzundan başlayarak kabloları döşer. Tek bir yasağı vardır: Eğer döşeyeceği kablo iki köy arasında zaten var olan bir bağlantıyı kapatıp yuvarlak bir döngü (cycle) oluşturuyorsa o kabloyu pas geçer.

Dijkstra ve Prim Farkı

- **Dijkstra:** Başlangıç noktasından diğer tüm noktalara olan **toplam yol uzunluğunu** minimize eder (Bireysel mesafe odaklıdır).
- **Prim (MST):** Sistemin kurulumundaki **toplam harcanan kablo/kenar maliyetini** minimize eder (Bütünsel maliyet odaklıdır).